

INF780

Verificación y Validación de Software

GUÍA DE LABORATORIO**Nro. 5****Pruebas de API con Postman***Aplicado al proyecto Movies API (NestJS + TypeORM + PostgreSQL)*

Materia	INF780 — Verificación y Validación de Software
Carrera	Ingeniería Informática
Universidad	Universidad Autónoma Tomás Frías
Docente	M. Sc. Huáscar Fedor Gonzales Guzmán
Repositorio base	github.com/HuascarFedor/INF780-MoviesApi (tag v1.0)
Tecnologías	Postman · Node.js · GitHub Actions

Potosí — Bolivia

1. Objetivos

1.1 Objetivo general

Diseñar y ejecutar una suite de pruebas funcionales sobre la API REST Movies utilizando Postman como cliente HTTP.

1.2 Objetivos específicos

- Construir requests manuales para cada uno de los cinco endpoints CRUD del recurso movies.
- Organizar los requests en una colección estructurada con variables de entorno y de colección.
- Escribir tests automatizados con la librería pm.* cubriendo los códigos 200, 201, 400, 404 y 422.

2. Marco teórico

2.1 Postman

Postman es una plataforma de desarrollo de APIs que permite construir, documentar y probar peticiones HTTP de manera interactiva. Sus elementos principales son:

- **Request:** una petición HTTP individual con método, URL, headers y body.
- **Collection:** agrupación lógica de requests; permite ejecución secuencial y comparte variables.
- **Environment:** conjunto de variables intercambiables (por ejemplo, dev y producción) que se inyectan en URLs y bodies con la sintaxis `{{variable}}`.
- **Pre-request script:** código JavaScript que corre antes del request (útil para preparar tokens, fechas, payloads dinámicos).
- **Tests:** código JavaScript que corre después del request y aplica aserciones sobre la respuesta usando `pm.test()` y `pm.expect()`.

2.2 La librería pm.*

Postman expone un objeto global pm dentro del editor de tests con los siguientes miembros relevantes:

Miembro	Propósito
<code>pm.response.code</code>	Código HTTP de la respuesta.
<code>pm.response.json()</code>	Parsea el body como JSON.
<code>pm.response.headers</code>	Acceso a los headers de respuesta.
<code>pm.test(name, fn)</code>	Define un caso de prueba con nombre y función.
<code>pm.expect(value)</code>	Aserciones estilo Chai (to.equal, to.have.property, to.be.an, etc.).

Miembro	Propósito
<code>pm.environment.set</code>	Persiste un valor en el entorno activo.
<code>pm.collectionVariables.set</code>	Persiste un valor en la colección (no requiere entorno seleccionado).

2.4 Pirámide de pruebas y ubicación de Postman

En la pirámide clásica, las pruebas con Postman ocupan el nivel de pruebas de API o de contrato: se ubican por encima de las pruebas unitarias y de integración del backend, y por debajo de las pruebas end-to-end de interfaz gráfica. Validan el comportamiento observable de la API tal como lo verá un cliente externo, sin importar la implementación interna.

3. Recurso bajo prueba: Movies API

3.1 Endpoints disponibles

Método	Endpoint	Descripción
POST	/movies	Crear una película. Body: CreateMovieDto.
GET	/movies	Listar todas las películas.
GET	/movies/:id	Obtener una película por UUID.
PATCH	/movies/:id	Actualizar parcialmente una película. Body: UpdateMovieDto.
DELETE	/movies/:id	Eliminar una película por UUID.

3.2 Reglas de validación de CreateMovieDto

Las reglas de validación están aplicadas con class-validator y se mapean a respuestas HTTP 422 cuando fallan. Conocerlas es indispensable para diseñar los casos de prueba negativos.

Campo	Tipo	Restricciones	Error → 422 si
title	string	obligatorio, max 255	vacío, no string, > 255 chars
director	string	obligatorio, max 150	vacío, no string, > 150 chars
genre	enum Genre	valor del enum	valor fuera del enum
year	int	$1888 \leq \text{year} \leq 2030$	decimal, < 1888 o > 2030
rating	number	$0 \leq \text{rating} \leq 10$, 1 dec.	negativo, > 10 o > 1 decimal
synopsis	string?	opcional	no string (cuando se envía)

3.3 Códigos de respuesta esperados

Código	Significado	Cuándo se produce
201	Created	POST exitoso.
200	OK	GET, PATCH y DELETE exitosos.
400	Bad Request	UUID con formato inválido (ParseUUIDPipe).
404	Not Found	UUID válido pero la película no existe.
422	Unprocessable Entity	Body no cumple las reglas del DTO.

4. Preparación del entorno

4.1 Requisitos previos

- Node.js 20.x o superior y npm.
- PostgreSQL 14+ corriendo localmente o en contenedor.
- Postman Desktop (versión 11 o superior).
- Cuenta de GitHub con acceso al repositorio del estudiante.

4.2 Clonar y levantar el proyecto

Clonar el repositorio base

Desde una terminal:

1

```
git clone https://github.com/HuascarFedor/INF780-MoviesApi.git
cd INF780-MoviesApi
git checkout v1.0
git checkout -b lab5-postman
```

Instalar dependencias

2

```
npm install
```

Configurar variables de entorno

Copiar .env.example a .env y ajustar credenciales de PostgreSQL.

3

```
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=postgres
DB_PASSWORD=postgres
DB_DATABASE=movies_db
PORT=3000
```

4

Levantar el API

```
npm run start:dev
```

Verificar que la salida muestre el listado de rutas mapeadas y el puerto 3000.

Verificación. En el navegador, abrir <http://localhost:3000/movies> y comprobar que devuelve [] (lista vacía) o un arreglo de películas existentes.

5

Instalar Newman globalmente

```
npm install -g newman  
npm install -g newman-reporter-htmlextra  
newman --version
```

La última línea debe imprimir la versión instalada (6.x o superior).

Importante. Las pruebas se ejecutarán contra la misma base de datos de desarrollo. Cada corrida genera y elimina películas de prueba, por lo que el estado final de la BD queda consistente. Si prefieres aislamiento total, puedes apuntar el .env a otra base antes de correr Newman.

5. Parte 1 — Requests manuales en Postman

El primer paso es ejecutar cada endpoint de forma interactiva para familiarizarse con la API y observar las respuestas reales.

5.1 Crear el workspace y la colección

1 Abrir Postman y crear un workspace

File → New → Workspace → "INF780 — Lab 5". Tipo: Personal.

2 Crear la colección

Botón "+" en la barra lateral → New Collection → nombre: Movies API. En la pestaña Variables, agregar la variable de colección movie_id (sin valor inicial).

3 Crear el environment

Engranaje superior derecho → Environments → Create. Nombre: Local. Agregar las siguientes variables:

Variable	Initial value	Current value
base_url	http://localhost:3000	http://localhost:3000
valid_uuid	(vacío)	(vacío)

Activar el environment Local desde el dropdown superior derecho.

5.2 Crear los 5 requests base

Dentro de la colección Movies API, agregar los siguientes requests con sus configuraciones exactas:

Request 1. POST Crear película

Method: POST · URL: {{base_url}}/movies · Body: raw / JSON

```
{
  "title": "Inception",
  "director": "Christopher Nolan",
  "genre": "sci-fi",
  "year": 2010,
  "rating": 8.8,
  "synopsis": "A thief who steals corporate secrets through dream-sharing technology."
}
```

En la pestaña Tests, agregar este script para capturar el id de la película creada:

```
// Capturar el id devuelto y guardarlo como variable de colección  
const data = pm.response.json();  
if (data && data.id) {  
    pm.collectionVariables.set("movie_id", data.id);  
    console.log("movie_id guardado:", data.id);  
}
```

Request 2. GET Listar películas

Method: GET · URL: {{base_url}}/movies

Request 3. GET Obtener por id

Method: GET · URL: {{base_url}}/movies/{{movie_id}}

Request 4. PATCH Actualizar parcialmente

Method: PATCH · URL: {{base_url}}/movies/{{movie_id}} · Body: raw / JSON

```
{  
  "rating": 9.0,  
  "synopsis": "Updated synopsis after rewatching the movie."  
}
```

Request 5. DELETE Eliminar por id

Method: DELETE · URL: {{base_url}}/movies/{{movie_id}}

Ejecutar en orden. Ejecutar los requests en el orden 1 → 2 → 3 → 4 → 5. El POST (Request 1) llena la variable movie_id, que los Requests 3, 4 y 5 reutilizan.

6. Parte 2 — Tests automatizados con pm.*

Una vez verificado que los requests funcionan manualmente, agregamos aserciones que conviertan cada ejecución en una prueba reproducible. Cubriremos cuatro categorías: happy path, validación 422, no encontrado 404 y formato de UUID inválido 400.

6.1 Resumen de casos de prueba

#	Categoría	Request	Esperado
01	Happy path	POST /movies (válido)	201
02	Happy path	GET /movies	200
03	Happy path	GET /movies/:id (existente)	200
04	Happy path	PATCH /movies/:id (existente)	200
05	Happy path	DELETE /movies/:id (existente)	200
06	Validación	POST con title vacío	422
07	Validación	POST con title > 255 chars	422
08	Validación	POST con director vacío	422
09	Validación	POST con genre fuera del enum	422
10	Validación	POST con year = 1800	422
11	Validación	POST con year = 2050	422
12	Validación	POST con rating = 11	422
13	Validación	POST con rating = 8.85	422
14	No encontrado	GET con UUID inexistente	404
15	No encontrado	PATCH con UUID inexistente	404
16	No encontrado	DELETE con UUID inexistente	404
17	UUID inválido	GET con id = 'abc-123'	400
18	UUID inválido	PATCH con id = 'abc-123'	400
19	UUID inválido	DELETE con id = 'abc-123'	400

6.2 Bloque A — Happy path (5 tests)

Test 01. POST /movies con datos válidos → 201

Pegar en la pestaña Tests del Request 1 (POST):

```
pm.test("Status code is 201 Created", function () {  
  pm.response.to.have.status(201);  
});
```

```
pm.test("Response is JSON object", function () {
  pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

pm.test("Response has UUID id", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("id");
  pm.expect(body.id).to.match(
    /^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$/i
  );
});

pm.test("Returned data matches sent payload", function () {
  const body = pm.response.json();
  pm.expect(body.title).to.eql("Inception");
  pm.expect(body.director).to.eql("Christopher Nolan");
  pm.expect(body.genre).to.eql("sci-fi");
  pm.expect(body.year).to.eql(2010);
  pm.expect(parseFloat(body.rating)).to.eql(8.8);
});

pm.test("Timestamps are present", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("createdAt");
  pm.expect(body).to.have.property("updatedAt");
});

// Persistir el id para los siguientes requests
const data = pm.response.json();
pm.collectionVariables.set("movie_id", data.id);
```

Test 02. GET /movies → 200

Pestaña Tests del Request 2 (GET listar):

```
pm.test("Status code is 200 OK", function () {
  pm.response.to.have.status(200);
});

pm.test("Response is an array", function () {
  pm.expect(pm.response.json()).to.be.an("array");
});
```

```
pm.test("Array contains the created movie", function () {  
  const body = pm.response.json();  
  const id = pm.collectionVariables.get("movie_id");  
  const found = body.find(m => m.id === id);  
  pm.expect(found, "movie_id should be present in the  
list").to.not.be.undefined;  
});
```

Test 03. GET /movies/:id → 200

```
pm.test("Status code is 200 OK", function () {  
  pm.response.to.have.status(200);  
});  
  
pm.test("Returned movie matches stored id", function () {  
  const body = pm.response.json();  
  const id = pm.collectionVariables.get("movie_id");  
  pm.expect(body.id).to.eql(id);  
  pm.expect(body.title).to.eql("Inception");  
});
```

Test 04. PATCH /movies/:id → 200

```
pm.test("Status code is 200 OK", function () {  
  pm.response.to.have.status(200);  
});  
  
pm.test("Rating was updated", function () {  
  const body = pm.response.json();  
  pm.expect(parseFloat(body.rating)).to.eql(9.0);  
});  
  
pm.test("Synopsis was updated", function () {  
  const body = pm.response.json();  
  pm.expect(body.synopsis).to.include("rewatching");  
});  
  
pm.test("updatedAt is more recent than createdAt", function () {  
  const body = pm.response.json();  
  const created = new Date(body.createdAt).getTime();  
  const updated = new Date(body.updatedAt).getTime();  
  pm.expect(updated).to.be.at.least(created);  
});
```

```
});
```

Test 05. DELETE /movies/:id → 200

```
pm.test("Status code is 200 OK", function () {  
    pm.response.to.have.status(200);  
});  
  
pm.test("Response body is empty or null", function () {  
    pm.expect(pm.response.text()).to.satisfy(t => t === "" || t === "null" ||  
    t.length < 5);  
});
```

6.3 Bloque B — Validación 422 (8 tests)

Para cada caso, duplicar el Request 1 (POST) en una subcarpeta llamada Validation y modificar el body. Renombrar cada copia con el número de test. El bloque de tests común para todos es:

```
pm.test("Status code is 422 Unprocessable Entity", function () {
  pm.response.to.have.status(422);
});

pm.test("Response has validation message array", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("message");
  pm.expect(body.message).to.be.an("array");
  pm.expect(body.message.length).to.be.greaterThan(0);
});

pm.test("Error name is Unprocessable Entity", function () {
  const body = pm.response.json();
  pm.expect(body.error).to.eql("Unprocessable Entity");
});
```

Bodies a usar en cada caso:

Test 06. title vacío

```
{
  "title": "",
  "director": "Christopher Nolan",
  "genre": "sci-fi",
  "year": 2010,
  "rating": 8.8
}
```

Aserción adicional específica:

```
pm.test("Validation error mentions title", function () {
  const messages = pm.response.json().message.join(" ");
  pm.expect(messages.toLowerCase()).to.include("title");
});
```

Test 07. title con > 255 caracteres

En la pestaña Pre-request Script del request, generar dinámicamente un título largo:

```
pm.variables.set("longTitle", "A".repeat(300));
```

Body:

```
{
  "title": "{{longTitle}}",
  "director": "Christopher Nolan",
  "genre": "sci-fi",
  "year": 2010,
  "rating": 8.8
}
```

Test 08. director vacío

```
{
  "title": "Inception",
  "director": "",
  "genre": "sci-fi",
  "year": 2010,
  "rating": 8.8
}
```

Test 09. genre fuera del enum

```
{
  "title": "Inception",
  "director": "Christopher Nolan",
  "genre": "musical",
  "year": 2010,
  "rating": 8.8
}
```

Test 10. year = 1800 (menor a 1888)

```
{
  "title": "Old Movie",
  "director": "Unknown",
  "genre": "drama",
  "year": 1800,
  "rating": 6.0
}
```

Test 11. year = 2050 (mayor a 2030)

```
{  
  "title": "Future Movie",  
  "director": "Unknown",  
  "genre": "sci-fi",  
  "year": 2050,  
  "rating": 7.5  
}
```

Test 12. rating = 11 (mayor a 10)

```
{  
  "title": "Overrated",  
  "director": "Anyone",  
  "genre": "comedy",  
  "year": 2020,  
  "rating": 11  
}
```

Test 13. rating con dos decimales

```
{  
  "title": "Two Decimals",  
  "director": "Anyone",  
  "genre": "drama",  
  "year": 2020,  
  "rating": 8.85  
}
```

6.4 Bloque C — No encontrado 404 (3 tests)

Para los tres casos se utiliza un UUID válido en formato pero garantizado a no existir en la base de datos. Crear una variable de entorno `nonexistent_uuid` con el valor:

```
00000000-0000-4000-8000-000000000000
```

Bloque de tests común:

```
pm.test("Status code is 404 Not Found", function () {
  pm.response.to.have.status(404);
});

pm.test("Response has not found message", function () {
  const body = pm.response.json();
  pm.expect(body.message).to.include("not found");
});

pm.test("Error field is Not Found", function () {
  const body = pm.response.json();
  pm.expect(body.error).to.eql("Not Found");
});
```

Test	Method	URL
14	GET	{{base_url}}/movies/{{nonexistent_uuid}}
15	PATCH	{{base_url}}/movies/{{nonexistent_uuid}}
16	DELETE	{{base_url}}/movies/{{nonexistent_uuid}}

Para el PATCH (Test 15), incluir un body válido mínimo:

```
{ "rating": 7.5 }
```

6.5 Bloque D — UUID con formato inválido 400 (3 tests)

Crear una variable de entorno `invalid_uuid` con el valor:

```
abc-123
```

Bloque de tests común:

```
pm.test("Status code is 400 Bad Request", function () {
```

```
pm.response.to.have.status(400);
});

pm.test("Response mentions UUID validation", function () {
  const body = pm.response.json();
  const messages = Array.isArray(body.message) ? body.message.join(" ") :
body.message;
  pm.expect(messages.toLowerCase()).to.satisfy(m =>
    m.includes("uuid") || m.includes("validation") || m.includes("format")
  );
});
```

Test	Method	URL
17	GET	{{base_url}}/movies/{{invalid_uuid}}
18	PATCH	{{base_url}}/movies/{{invalid_uuid}}
19	DELETE	{{base_url}}/movies/{{invalid_uuid}}

7. Parte 3 — Ejecución con Collection Runner

Antes de pasar a la línea de comandos, validamos toda la suite desde la interfaz gráfica de Postman. El orden de ejecución importa: el POST del bloque Happy path debe correr antes de los tests 03 y 04, que dependen de `movie_id`.

7.1 Estructura final de la colección

La colección Movies API debe quedar organizada con esta jerarquía:

```
Movies API/  
├─ 01 Happy path/  
│   ├── 01 POST create  
│   ├── 02 GET all  
│   ├── 03 GET by id  
│   ├── 04 PATCH update  
│   └─ 05 DELETE  
├─ 02 Validation 422/  
│   ├── 06 title empty  
│   ├── 07 title too long  
│   ├── 08 director empty  
│   ├── 09 genre invalid  
│   ├── 10 year too low  
│   ├── 11 year too high  
│   ├── 12 rating too high  
│   └─ 13 rating too many decimals  
├─ 03 Not found 404/  
│   ├── 14 GET nonexistent  
│   ├── 15 PATCH nonexistent  
│   └─ 16 DELETE nonexistent  
└─ 04 Invalid UUID 400/  
    ├── 17 GET invalid uuid  
    ├── 18 PATCH invalid uuid  
    └─ 19 DELETE invalid uuid
```

7.2 Ejecutar el Runner

1**Abrir el Runner**

Click derecho sobre la colección Movies API → Run collection.

2**Configurar la corrida**

- Environment: Local
- Iterations: 1

- Delay: 0 ms
- Marcar Save responses para inspeccionar luego

Ejecutar y revisar

Run Movies API. La barra superior debe mostrar 19 tests pasados (verde). Si alguno falla, abrir el request culpable y revisar la pestaña Test Results.

3

Resultado esperado. All 19 tests passed. Avg response time menor a 200 ms en entorno local.

8. Bibliografía

Postdot Technologies. (2025). *Postman Learning Center: Writing tests in Postman*. Postman, Inc. <https://learning.postman.com/docs/writing-scripts/test-scripts/>

Postdot Technologies. (2025). *Newman: Command-line companion for Postman*. <https://learning.postman.com/docs/collections/using-newman-cli/command-line-integration-with-newman/>

GitHub, Inc. (2025). *GitHub Actions Documentation: Using service containers*. <https://docs.github.com/en/actions/using-containerized-services/about-service-containers>

Mysliwiec, K. (2025). *NestJS Documentation: Validation*. <https://docs.nestjs.com/techniques/validation>

Fielding, R. T., & Reschke, J. (2022). *RFC 9110: HTTP Semantics*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>

Crispin, L., & Gregory, J. (2014). *More Agile Testing: Learning Journeys for the Whole Team*. Addison-Wesley Professional.